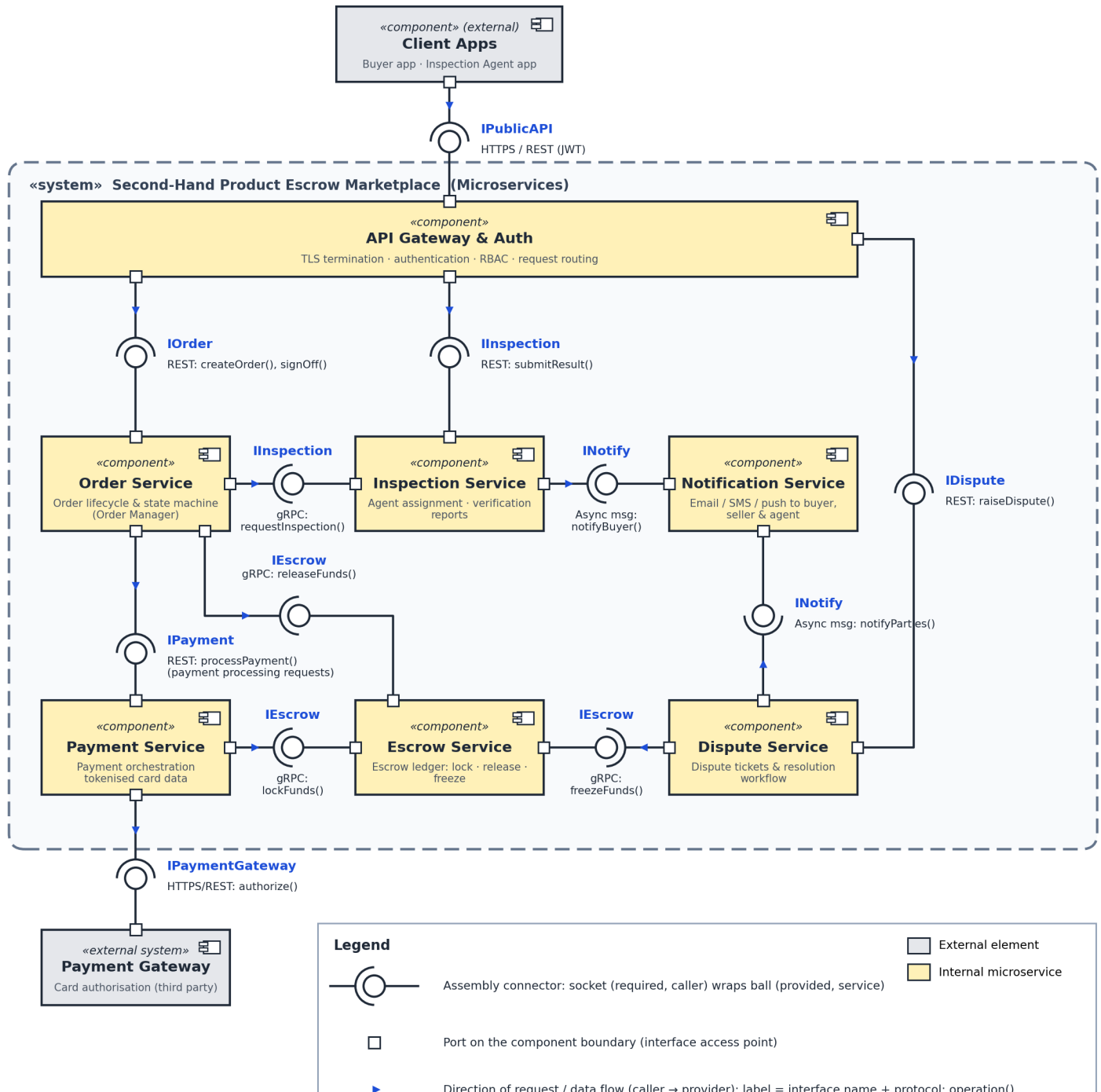


Lab 3 - UML Component Diagram

Second-Hand Product Escrow Marketplace | Architecture: Microservices | Problem #39 (Retail, E-Commerce & Finance)



Lab 3 – Architectural Justification

Samarth M H | SRN: PES1UG24CS407 | Problem #39 – Second-Hand Product Escrow Marketplace (Retail, E-Commerce & Finance)

Architecture Selection

We chose Microservices Architecture for the Second-Hand Product Escrow Marketplace System. The system is decomposed into seven independently deployable services behind an API Gateway: *Order*, *Payment*, *Escrow*, *Inspection*, *Dispute* and *Notification*, plus an external Payment Gateway. Each service owns its data and exposes a narrow provided interface (IOrder, IPayment, IEscrow, IInspection, IDispute, INotify), as shown in the component diagram.

Style Comparison (Step 2)

Style	Fit for this scenario	Verdict
Layered	Simple, but Payment, Escrow, Inspection and Dispute would share one business layer and one database; a change or failure in one part affects all, and it can only be scaled as a whole.	Rejected
Client-Server	A single central server is a single point of failure and a bottleneck – unacceptable when money is held in escrow and a dispute must freeze funds immediately.	Rejected
Microservices	Matches the multi-party workflow (buyer, seller, inspection agent, payment gateway); gives isolation, independent scaling and deployment. Cost: network latency and distributed consistency.	Selected

Two Reasons for the Choice

1. The scenario splits naturally into independent business capabilities. The escrow workflow (FR-001 to FR-005) is a chain of separate responsibilities: lock payment, assign an inspection, obtain buyer sign-off, release or dispute. Each maps to one service with its own lifecycle. For example, the Payment Service hides the card processor behind IPaymentGateway, so the gateway can be replaced without touching Order or Escrow, and inspection rules can change without redeploying the escrow ledger.

2. Fault isolation protects money-critical paths. If Notification or Inspection is slow or down, the Escrow ledger and the Dispute service keep working, so a buyer's dispute still freezes the funds (NFR-001). Failures in low-criticality components never corrupt an escrow balance, and non-critical calls (notifications) are asynchronous so they cannot block a purchase.

Security Advantage

Security-sensitive logic is isolated behind small, well-defined interfaces. The API Gateway is the single entry point and enforces authentication and role-based access (Buyer, Inspection Agent, Admin), satisfying NFR-002. Card data stays inside the Payment Service (tokenised), which limits the compliance and breach scope. Only Payment can *lockFunds*, only Order can *releaseFunds* after valid buyer sign-off, and only Dispute can *freezeFunds*; services talk over mutually authenticated TLS and each database is encrypted at rest. A compromised service therefore cannot release funds on its own.

Performance Benefit

Services scale independently: Order and Escrow (high-volume, transactional) can run many replicas, while Inspection (human-paced, bursty) needs few. The dispute freeze is a single small call to the Escrow Service, so funds are frozen immediately (NFR-001) regardless of load elsewhere. Asynchronous messaging for notifications keeps checkout latency low. The added network overhead and eventual consistency are managed with idempotent requests and a saga-style order flow.